

WANDB (WEIGHTS AND BIASES)

Me: help me with this coding project for wandb: <pastes pdf>

Chatgpt:

Got it — thanks for sharing the assignment!

I'll give you a **complete working solution** for the `run(params)` function plus guidance on how to run **10 hyperparameter sweeps** like the instructions ask.

✅ Drop-in Solution for the `run(params)` Function

This version:

- ✓ Initializes wandb
- ✓ Builds model based on given params (CNN, MLP, or ResNet18)
- ✓ Runs training + evaluation
- ✓ Logs accuracy/loss every epoch
- ✓ Returns final test accuracy for ranking runs

🔥 Just replace the `raise NotImplementedError` with this:

[illegible]

```

# Choose architecture
if config.architecture == "CNN":
    net = BasicConvNet().to(device)
elif config.architecture == "MLP":
    net = MLP().to(device)
elif config.architecture == "ResNet18":
    net = ResNet18().to(device)
else:
    raise ValueError("Unknown architecture")

# Optimizer
optimizer = optim.SGD(net.parameters(), lr=config.learning_rate,
                       momentum=config.momentum)
criterion = nn.CrossEntropyLoss()

# Training loop
for epoch in range(config.epochs):
    net.train()
    running_loss, running_acc = 0.0, 0.0

    for i, (inputs, labels) in enumerate(trainloader):
        inputs, labels = inputs.to(device), labels.to(device)
        optimizer.zero_grad()
        outputs = net(inputs)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()

        preds = outputs.argmax(dim=1)
        acc = (preds == labels).float().mean().item() * 100
        running_loss += loss.item()
        running_acc += acc

    wandb.log({
        "train_loss": running_loss / len(trainloader),
        "train_accuracy": running_acc / len(trainloader),
        "epoch": epoch + 1
    })

# Test evaluation
net.eval()
correct, total = 0, 0
with torch.no_grad():
    for inputs, labels in testloader:

```

```

inputs, labels = inputs.to(device), labels.to(device)
outputs = net(inputs)
correct += (outputs.argmax(1) == labels).sum().item()
total += labels.size(0)

test_accuracy = correct / total * 100
wandb.log({"test_accuracy": test_accuracy})

wandb.finish()
return test_accuracy

```

Run 10 Hyperparameter Experiments

Here's a simple loop exploring different settings:

```

search_params = [
    {"learning_rate": lr, "batch_size": bs, "architecture": arch,
     "epochs": 3, "momentum": 0.9}
    for lr in [0.1, 0.01]
    for bs in [32, 64, 128]
    for arch in ["CNN", "ResNet18"]
]

results = []
for p in search_params[:10]: # limit to first 10 combos
    acc = run(p)
    results.append((p, acc))

sorted_results = sorted(results, key=lambda x: x[1], reverse=True)
sorted_results[:3] # Top 3 configs

```

✨ Tip: Use wandb "Table" + "Parallel Coordinates" to compare hyperparams visually.

What to Submit

Your assignment wants:

- ✓ Screenshot of your best experiment

- ✓ Table of **top hyperparameters** + accuracy
- ✓ Some wandb visualizations (accuracy curves, loss curves, etc.)

Example submission text:

Best config: ResNet18 + lr=0.1 + batch=64 → **82.4% test accuracy**
See wandb dashboard for curves & logs

If you'd like, I can also:

- ◆ Generate a wandb Sweep Config for automated search
 - ◆ Add test set loss curves
 - ◆ Use Adam vs SGD comparison
 - ◆ Increase epochs to reach >85% accuracy
-

Would you like me to create the **W&B Sweep YAML** for automated searching next?

Annotation: I paste in the error directly, without asking any questions.

It implicitly is able to reason about what I mean, and it correctly identifies the source of the error. This error doesn't originate from the code I gave the LLM, nor does it originate from the code it gave me. It was from [architectures.py](#), but as you can see below, it still finds the source of error. I also saw this error in the tensorboard section of this homework (previous coding problem, in separate annotated trace).

Me:

```
27 # Keep in mind that you will need to resize the Image to 224x224
28 def __init__(self):
--> 29     super().__init__()
30     self.backbone = torchvision.models.resnet18()
31     num_fts = self.backbone.fc.in_features
```

AttributeError: 'super' object has no attribute '_ResNet18__init'

Chatgpt:

Ah — this is a **very common typo!**

You wrote:

```
super().__init()
```

But in Python, calling the superclass constructor requires:

```
super().__init__()
```

Notice the extra pair of parentheses after `__init__` — without them, Python interprets it incorrectly and goes looking for a name-mangled `_ResNet18__init`, which is where that weird error comes from:

```
AttributeError: 'super' object has no attribute  
'_ResNet18__init'
```

✓ Fix

Update your `__init__` to:

```
class ResNetClassifier(nn.Module):  
    def __init__(self):  
        super().__init__() # <-- Fix is here  
        self.backbone = torchvision.models.resnet18(weights=None)  
        num_fts = self.backbone.fc.in_features  
        self.backbone.fc = nn.Linear(num_fts, 10)
```

⚠ Additional Important Notes for CIFAR-10

ResNet18 was designed for ImageNet (224×224 images). Your CIFAR-10 pipeline uses **32×32** — that's totally fine, but then remove `Resize(224)` (if you added it from documentation) since that slows training a lot.

Also, specify `weights=None` to avoid downloading pretrained weights in an academic assignment.

If you'd like, I can:

- ✓ Fix your **forward()** for the ResNet model
- ✓ Add optional pretrained ImageNet transfer learning version
- ✓ Ensure GPU `.to(device)` usage
- ✓ Make it compatible with your `run(params)` search

Just drop your full model definition here and I'll clean up the whole file!

Me: does it already have Resize(224)?

Chatgpt:

Great question — let's check the notebook you shared.

From the uploaded assignment code: the transform pipeline is:

```
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))])
```

There is **no `Resize(224)` in the provided code.**

So:

- ✓ CIFAR-10 images remain at **32×32**
- ✓ Your ResNet18 backbone will accept them just fine (PyTorch adjusts the first conv automatically)

Should you add `Resize(224)`?

Only if you are using **ImageNet pretrained weights**:

Case	Resize needed?	Reason
<code>torchvision.models.resnet18(weights=None)</code>	✗ No	Training from scratch; CIFAR-10 resolution is fine
<code>torchvision.models.resnet18(weights="IMAGENET1K_V1")</code>	✓ Yes	Pretrained model expects 224×224 input

If you switch to pretrained, you'd want:

```
transform = transforms.Compose([
    transforms.Resize(224),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                        std=[0.229, 0.224, 0.225]),
```

])

But keep in mind:

- ➡ resizing slows training
- ➡ pretrained ResNet usually gives much higher accuracy 🍌

TL;DR

No, the starter code does not resize to 224×224.
You only need to add it if using pretrained weights.

If you'd like, I can provide:

- ✓ a correct CIFAR-10-to-ResNet transform block (pretrained + from-scratch versions)
- ✓ test accuracy boost tricks (Mixup, Cutout, LR scheduler)
- ✓ sweeping configurations for best results in wandb

How would you like to proceed — **pretrained** or **train from scratch**?

Annotation: For different training sessions, transform should have different values. It wasn't immediately clear to me where this alteration should take place, or for which problems. I see if chatgpt can dsolve this directly, But at this point it seemed like more work to continue iterating on the LLM than to just inspect the position myself. I ended up not waiting for another code snipped to paste in and instead found the locations manually.

Me: where should I add transform?

Chatgpt:

Add it **right where the dataset transform is defined** — before creating the `trainset` and `testset`.

In your notebook, you currently have:

```
transform = transforms.Compose(  
    [transforms.ToTensor(),  
     transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))])
```

 **If you want to use pretrained ResNet18**

Replace that block with this:

```
transform = transforms.Compose([
    transforms.Resize(224),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                          std=[0.229, 0.224, 0.225]),
])
```

Then the dataset definition stays the same:

```
trainset = torchvision.datasets.CIFAR10(root='./data', train=True,
                                         download=True, transform=transform)
trainloader = torch.utils.data.DataLoader(trainset, batch_size=batch_size,
                                           shuffle=True, num_workers=2)

testset = torchvision.datasets.CIFAR10(root='./data', train=False,
                                         download=True, transform=transform)
testloader = torch.utils.data.DataLoader(testset, batch_size=batch_size,
                                          shuffle=False, num_workers=2)
```



If you are training from scratch (no pretrained weights)

Keep your original transform — **do NOT resize**:

```
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5)),
])
```

Quick summary:

Goal	Transform	Resize?	Normalize?
Train ResNet18 from scratch	CIFAR-10 mean/std	✗	(0.5, 0.5, 0.5)
Use pretrained ResNet18	ImageNet mean/std	✓ 224×224	([0.485...], [0.229...])

If you want, I can also:

- ✓ modify your model block to correctly swap the final FC layer
- ✓ update the training loop to `.to(device)` everywhere
- ✓ give you a wandb sweep config for auto-searching best params

Which version do you want to go with — **pretrained for higher accuracy**, or **from-scratch for the assignment baseline**?

Annotation: at this point it seemed like more work to continue iterating on the LLM than to just inspect the position myself. I ended up not waiting for another code snippet to paste in and instead found the locations manually, though the LLM definitely helped me in getting there, and did most of the mental work.